

Distributed Cuckoo Filter on ESP32 Over ESP-NOW for Resource-Constrained IoT Edge Networks

Ayesha Kashif
Institute of Business Administration
Karachi, Pakistan
29155@iba.edu.pk

Syed Hamza Ahsan
Institute of Business Administration
Karachi, Pakistan
28903@iba.edu.pk

Kabeer Jafri
Institute of Business Administration
Karachi, Pakistan
29027@iba.edu.pk

Qazi Muhammad Waiz
Institute of Business Administration
Karachi, Pakistan
28982@iba.edu.pk

Abstract—Modern IoT edge networks increasingly require fast, memory-efficient set-membership verification that can operate without relying on a central server. This paper presents the design, implementation, and evaluation of a Cuckoo Filter distributed across multiple ESP32 microcontrollers communicating over ESP-NOW, a connectionless 802.11 MAC-layer protocol with minimal framing overhead. The filter is partitioned into fixed 256-bucket segments per slave node, with each bucket holding four 8-bit fingerprints derived using MurmurHash2. A master node coordinates all INSERT, DELETE, and LOOKUP operations by routing requests to the appropriate slave based on deterministic hash-derived bucket assignments. Fault tolerance is handled through an idempotent deduplication cache maintained on each slave, which prevents duplicate execution of retransmitted requests.

The transport layer was redesigned and migrated from WebSocket over TCP to ESP-NOW in order to eliminate per-message framing costs and avoid the per-connection heap overhead that was pushing the master node close to its memory limit. Physical hardware testing on a one-master plus one-slave configuration measured end-to-end operation latency between 2 and 5 milliseconds and sustained throughput of roughly 500 to 800 operations per second, with a false positive rate of 1.61 percent at 50 percent occupancy. A 32-case test suite covering unit, integration, and benchmark scenarios compiles and runs entirely on a native desktop build without requiring physical hardware. The architecture cleanly supports 1, 2, 4, or 8 slave nodes and incorporates automatic load rebalancing triggered at 85 percent bucket occupancy.

Index Terms—cuckoo filter, probabilistic data structures, ESP32, ESP-NOW, distributed set-membership, IoT edge computing, MurmurHash2, embedded systems, wireless mesh

I. INTRODUCTION

Embedded IoT deployments at the network edge increasingly need to make real-time decisions without waiting for a response from a remote server. Use cases such as sensor registration checks, anomaly detection, packet deduplication, and access control all boil down to a single fundamental question: is this item a member of a known set? These membership queries must resolve in single-digit milliseconds on hardware

with severely constrained memory and no persistent network connection.

The textbook solution is a hash table, but maintaining one on a microcontroller with under a megabyte of SRAM is impractical when the set can grow to thousands of elements. Bloom filters address the memory problem by representing set membership as a fixed-size bit array, achieving a tunable false positive rate at the cost of deletions, which require a space-expensive counting extension to support. The Cuckoo Filter, developed by Fan et al., solves the deletion problem by storing compact fingerprints in a bucketized cuckoo hash table. Each key occupies one of two hash-determined bucket locations, and the filter supports lookup, insertion, and deletion natively. At 8-bit fingerprints with four slots per bucket the false positive rate is approximately 3 percent, and the total memory footprint for a 256-bucket partition is around one kilobyte, well within the ESP32's 520 KB of available SRAM.

Distributing the filter across multiple nodes introduces a communication cost that can easily exceed the cost of the filter operation itself. Early versions of this project used WebSocket over TCP for inter-node communication, which added 8 to 14 bytes of framing to every message and required per-client connection state that was exhausting heap on the master node. ESP-NOW, Espressif's MAC-layer exchange protocol, eliminates both problems. Peers are registered once at boot using hardware MAC addresses, sends require no TCP handshake, and the protocol adds no application-layer framing at all.

This paper presents the full design, implementation, and evaluation of a distributed Cuckoo Filter running on a cluster of ESP32 boards communicating over ESP-NOW. The main contributions of this work are as follows. First, we designed a layered firmware architecture in which the transport, dispatcher, and filter modules are independently testable on a desktop without any hardware dependency. Second, we defined a packed binary wire protocol that keeps all operation frames under 250 bytes while carrying enough metadata for dedu-

plication, routing, and rebalancing. Third, we measured empirical latency, throughput, and false positive rate on physical ESP32 hardware and validated multi-slave behavior through the Wokwi simulator.

The rest of this paper is organized as follows. Section II reviews related work on probabilistic data structures and IoT communication protocols. Section III describes the system design and implementation in detail. Section IV reports our experimental results. Section V concludes with a discussion of limitations and directions for future work.

II. RELATED WORK

A. Probabilistic Set-Membership Data Structures

Set-membership filters form a well-studied family of space-efficient probabilistic data structures, and choosing the right one for a constrained embedded target requires understanding the trade-offs within that family.

Burton Bloom introduced the bit-array membership filter in 1970. A Bloom filter maps each set element into a shared bit array using k independent hash functions, and a query returns a false positive whenever all k bits happen to be set by prior insertions. The false positive rate is a function of the array size, the number of hash functions, and the number of inserted elements, and it can be tuned by adjusting these parameters at the cost of memory. Deletions are not supported in the basic design because clearing bits could invalidate other elements that share them. Counting Bloom filters extend the design by replacing each bit with a small counter, which allows decrements on deletion, but they require three to four times as much memory as the standard filter.

The quotient filter improves cache locality by encoding each fingerprint as a quotient and remainder stored in adjacent slots, avoiding the scattered memory accesses that hurt Bloom filter performance on cache-sensitive hardware. However, the insertion algorithm is more complex and the structure does not expose the two-location alternate-bucket property that the Cuckoo Filter relies on for its compact relocation logic.

Pagh and Rodler introduced cuckoo hashing as a general hash table design achieving worst-case $O(1)$ lookups by allowing each key to occupy one of two hash-determined locations and displacing occupants in a chain until an empty slot is found. Fan et al. adapted this to fingerprint-only storage, producing the Cuckoo Filter. The key insight is the alternate-bucket formula $i \oplus \text{hash}(f)$, which computes the second candidate bucket from only the fingerprint f and the current bucket index i , with no need to store or re-hash the original key. At 8 bits and four slots per bucket Fan et al. report a 3 percent false positive rate at up to 95 percent occupancy, with space efficiency exceeding equivalent Bloom filters. This is the configuration adopted by the present work.

B. Distributed Probabilistic Filters

Distributed Bloom filters have been studied in peer-to-peer overlay networks, where each node holds a partition of the global filter and queries propagate to the responsible shard. A consistent finding in that literature is that communication

cost dominates performance when the per-operation network round-trip exceeds the local filter operation time by an order of magnitude. Our system faces the same trade-off and addresses it by selecting the lowest-overhead wireless protocol available on the hardware and keeping operation payloads to 39 bytes per frame.

Partitioned filter designs have also appeared in content-centric networking contexts, where routers maintain compact forwarding information bases using Bloom filter variants. The partitioning strategy in those systems (range-based assignment by namespace prefix) is structurally similar to the hash-range assignment used here, though the scale and hardware differ substantially.

C. Communication Protocols for IoT Microcontrollers

MQTT and CoAP are the dominant application-layer protocols for constrained IoT devices. MQTT operates over TCP and requires a broker, while CoAP runs over UDP and is designed for request-response interactions between constrained devices. Both protocols add meaningful application-layer overhead and require infrastructure (broker or server) that may not be available in fully distributed edge deployments.

ESP-NOW is a Espressif-proprietary peer-to-peer protocol operating directly at the 802.11 MAC layer. Peers are registered by MAC address at startup, and subsequent sends are single API calls with no connection state or handshake. The payload limit is 250 bytes, delivery is best-effort with no built-in acknowledgment, and the protocol adds no framing bytes at all. For the short fixed-length frames in this system, ESP-NOW offers the best available trade-off between overhead and implementation complexity on the ESP32. Its main limitation, the lack of built-in acknowledgment, is handled at the application layer through sequence-numbered frames and a retransmission loop in the master.

III. METHODOLOGY

A. System Architecture Overview

The cluster consists of one master node and between one and eight slave nodes, all running on ESP32 microcontrollers. The master accepts commands from a developer over a 115200-baud serial terminal, computes hash-based routing decisions entirely in software, and dispatches packed binary frames over ESP-NOW to the appropriate slave. Slaves independently maintain their filter partitions, reply with ACK frames containing result codes and occupancy statistics, and send periodic heartbeat frames every two seconds regardless of whether any operation is in flight.

Bucket partitioning uses a static range assignment compiled into firmware at build time. With N slave nodes where N is one of 1, 2, 4, or 8, slave s owns the global bucket range $[s \cdot 256/N, (s + 1) \cdot 256/N - 1]$. The master computes the primary bucket index b_1 and the alternate bucket index $b_2 = b_1 \oplus \text{hash}(f)$ for each incoming key, determines which slave owns b_1 , and sends the operation frame to that slave. If the first bucket is full the master may also probe b_2 on the other slave. Peer MAC addresses are embedded in `config.h` and

registered with the ESP-NOW stack at boot; no runtime peer discovery is needed.

The software is organized into four independently testable modules. The `cuckoo_filter` module implements the single-node filter logic and has no platform dependency. The `slave_storage` module manages a heap-allocated pool of bucket blocks with a global index mapping. The `dispatcher` module handles message routing, deduplication, and reply generation on each slave. The `transport` module wraps all ESP-NOW API calls and is replaced by a no-op stub on native desktop builds so the rest of the code can be tested without hardware.

B. Cuckoo Filter Design and Parameters

Each slave instantiates one Cuckoo Filter with the configuration shown in Table I. The parameters were chosen to balance memory usage, false positive rate, and operational complexity on the ESP32 hardware.

TABLE I
SINGLE-NODE CUCKOO FILTER CONFIGURATION

Parameter	Value
Buckets per node	256
Slots per bucket	4
Fingerprint width	8 bits
Hash function	MurmurHash2 (32-bit)
Maximum kickout steps	500
Victim cache entries	1
RAM per partition	approximately 1 KB
Theoretical FPR at 50%	approximately 3%
Maximum safe occupancy	approximately 95%

Fingerprints are derived from the raw key bytes by applying MurmurHash2 with the seed `0xbc9f1d34` and extracting the low 8 bits of the result. A fingerprint value of zero is remapped to one because zero is used as the empty-slot sentinel in the bucket array. The alternate bucket is computed as $b_2 = (b_1 \oplus (f \times 0x5bd1e995)) \bmod 256$, which is the formula from the original Cuckoo Filter paper adapted for a power-of-two table size using bitwise masking.

When both candidate buckets are occupied and a new fingerprint must be inserted, the filter selects a random slot in one of the two buckets, evicts the occupant, places the new fingerprint there, and attempts to re-insert the evicted fingerprint into its own alternate bucket. This displacement chain runs for at most 500 iterations. If no empty slot is found within that limit, the last displaced fingerprint is stored in a one-entry victim cache and the insertion returns success to the caller. The victim cache entry participates in subsequent lookups and deletions; the filter is never left in an inconsistent state. This behavior mirrors the original Fan et al. design and bounds worst-case insertion time without the risk of an unbounded loop.

All bucket memory is allocated once at boot inside the `SlaveStorage` constructor using a pre-sized vector. No dynamic allocation occurs during filter operations. This is

important on the ESP32 because repeated small heap allocations during operation lead to fragmentation that degrades performance over time and can eventually cause allocation failures.

C. Wire Protocol

All inter-node communication uses packed C structures defined in `protocol/messages.h` and annotated with `__attribute__((packed))` to prevent the compiler from inserting alignment padding. Every frame is guaranteed to fit within the 250-byte ESP-NOW payload limit. Table II lists the frame types and their sizes.

TABLE II
WIRE PROTOCOL FRAME TYPES AND SIZES

Frame Type	Direction	Size (bytes)
INSERT / LOOKUP / DELETE	Master to slave	15
ACK	Slave to master	8
CHAIN_INSERT	Master to slave	16
CHAIN_RESULT	Slave to master	9
HEARTBEAT	Slave to master	13
BUCKET_BATCH_READ	Master to slave	47
BUCKET_BATCH_DATA	Slave to master	127
BUCKET_BATCH_WRITE	Master to slave	127
HARD_RESET	Master to slave	6

Every frame shares a common 6-byte header containing a 1-byte message type code, a 1-byte rolling sequence number, a 2-byte source node ID, and a 2-byte destination node ID. The sequence number is used by the slave’s deduplication cache to detect retransmissions. When a slave receives a frame whose (source ID, sequence number) pair is already in the cache, it discards the frame and replays the cached reply without re-executing the filter operation. The deduplication cache holds 64 entries in a circular buffer, which is sufficient to cover the retransmission window in all tested configurations.

The master sends each operation synchronously: it transmits a frame, waits up to one second for the matching ACK frame, and retransmits once on timeout. This simple stop-and-wait approach avoids the complexity of a sliding window and is adequate for a single-master single-slave or single-master multi-slave configuration where one outstanding request at a time is the normal case.

D. Kickout Chain Protocol

When an insertion requires displacing a fingerprint, the master coordinates a distributed kickout chain across multiple round trips. The chain begins when the initial CHAIN_INSERT to the primary bucket returns a STATUS_KICKED response along with the evicted fingerprint. The master then computes the alternate bucket for that evicted fingerprint, determines which slave owns it, and sends a new CHAIN_INSERT to that slave. This continues until either a free slot is found or the chain depth exceeds the configured maximum of five hops, at which point the insert returns STATUS_FULL.

Each CHAIN_INSERT frame carries a unique 32-bit transaction ID and a hop counter so that the deduplication cache can distinguish between separate insertion operations that happen to reuse the same sequence number (due to the 8-bit sequence number rolling over), and also so that slaves can detect and reject stale frames from an earlier interrupted chain.

E. Transport Layer Migration from WebSocket to ESP-NOW

The first two project milestones used WebSocket over TCP for all inter-node communication. Two separate problems motivated the decision to migrate to ESP-NOW in the third milestone.

The first problem was heap pressure. The ESP32's TCP stack maintains per-connection send and receive buffers on the heap. With the filter partition occupying roughly 1 KB and the firmware stack using additional heap, adding TCP connection state for even two or three simultaneous clients pushed available free heap below a safe threshold. The master began reporting allocation warnings during sustained test runs.

The second problem was framing overhead. WebSocket encapsulates each message in a frame with a 2-byte header for small payloads, growing to 8 to 14 bytes for larger ones, plus a mandatory 4-byte masking key for client-to-server frames. For the 15-byte operation payloads in this system, that framing adds 27 to 93 percent overhead. ESP-NOW eliminates both problems by operating directly at the MAC layer with no connection state and no framing bytes at all.

The migration was contained entirely within `transport/transport.cpp`, a 60-line file. All layers above, including the dispatcher, the filter logic, the routing table, and the serial CLI, remained completely unchanged. A preprocessor guard (`#ifndef NATIVE_BUILD`) skips the ESP-NOW API calls during native desktop builds, keeping the full test suite runnable on any laptop regardless of hardware availability.

F. Load Rebalancing Mechanism

Each slave reports its current occupancy as a percentage in every ACK frame and in every periodic heartbeat. The master tracks the reported occupancy for all registered slaves and triggers a rebalancing operation when any slave exceeds 85 percent occupancy or when the difference between the most-loaded and least-loaded slave exceeds 20 percentage points.

The rebalancing operation migrates a batch of up to 20 buckets from the overloaded slave to the least-loaded slave in three sequential round trips. First, the master sends a `BUCKET_BATCH_READ` command to the source slave, which replies with the fingerprint data for each requested bucket. Second, the master sends a `BUCKET_BATCH_WRITE` command to the destination slave with the received data. Third, after receiving a success ACK from the destination, the master sends a `BUCKET_BATCH_CLEAR` to the source to free those buckets. The three-step ordering guarantees that no fingerprint entries are lost if any individual frame is dropped: the destination receives the data before the source is cleared.

During migration, the affected buckets are marked as locked in the master's routing table. Any INSERT, LOOKUP, or DELETE operation that targets a locked bucket returns `STATUS_RETRY` to the caller, which the CLI displays as a RETRY result. This is a transient condition that resolves as soon as the migration completes, typically within three round trips. Rebalancing is rate-limited to at most once every 15 seconds to prevent oscillation when load is fluctuating near the threshold.

New slaves that connect to an existing cluster are detected through their first heartbeat frame. If a slave reports non-zero occupancy on first contact (indicating a reboot with retained RAM state), the master issues a `HARD_RESET` command, waits for an ACK, and then re-registers the slave with zero load. A new join with zero occupancy triggers a rebalancing pass to redistribute buckets onto the new node.

G. Testing Methodology

The test suite is split into four separate suites in the `test/` directory tree. The `test_cuckoo_filter` suite covers all single-node filter operations: basic insert, lookup, and delete; lookup of absent keys; repeated insertion of 100 distinct keys followed by lookup of all of them; filling to near-capacity and verifying that occupancy exceeds 80 percent; clear and reset behavior; and load percentage reporting accuracy.

The `test_e2e` suite simulates a full multi-slave cluster entirely in software within a single process. Each slave is represented by a live `SlaveStorage` and `Dispatcher` instance; the transport layer is replaced by a function pointer that delivers outgoing frames directly back into the test harness. This approach tests the complete message path, including deduplication, chain insert with kickout, bucket batch migration, and hard reset, without any radio hardware. Ten sequential test cases exercise routing initialization, basic insert-lookup-delete, multi-key load, try-only semantics, deduplication replay correctness, batch migration, locked bucket retry, hard reset, heavy load with FPR measurement, and slave unavailability propagation.

The `test_protocol` suite asserts on the exact binary layout of each packed struct, verifying that sizes and field byte offsets match the expected values on both ESP32 (ARM Cortex-M) and x86-64 (little-endian). This suite catches any accidental introduction of padding that would break cross-platform compatibility.

The `test_benchmark` suite measures false positive rate at a controlled occupancy level and records maximum achievable occupancy before the filter stops accepting new items.

All four suites compile and execute with `pio test -e native` and require no hardware. Hardware verification used two physical ESP32 boards connected over USB to a single development laptop. Multi-slave configurations beyond two boards were verified using the Wokwi simulation extension in VS Code.

IV. RESULTS AND DISCUSSION

A. False Positive Rate

The false positive rate measured by the native benchmark suite was **1.61%** (161 false positives out of 10,000 non-member queries) with 500 items inserted into a 1,024-capacity single-partition filter, corresponding to approximately 49 percent occupancy. This result falls below the theoretical ceiling of approximately 3.1 percent for an 8-bit fingerprint with four slots per bucket, consistent with the observation that Fan et al.’s formula represents a worst-case bound rather than a precise prediction. Hardware testing using the built-in FPR command reproduced a rate of approximately 1.8 percent under the same insertion count, confirming that the native build faithfully models physical filter behavior.

No significant deviation from the expected rate was observed. The slight variance between individual test runs is attributable to the pseudo-random seed used during insertions affecting which fingerprint collisions occur in the specific bucket configuration. Increasing the fingerprint width from 8 to 16 bits would reduce the FPR to under 0.1 percent at the cost of doubling RAM per partition from 1 KB to 2 KB, a trade-off that remains well within the ESP32’s 520 KB SRAM budget even for an 8-node cluster.

B. Maximum Occupancy

The benchmark suite confirmed that the filter can be loaded to **97.7 percent occupancy** (1,000 successful insertions into a 1,024-slot capacity) before reaching its displacement limit. This exceeds the 95 percent figure cited by Fan et al. and demonstrates that the victim-cache fallback effectively prevents early saturation. At this occupancy level, the false positive rate remained below 2 percent, confirming that performance does not degrade significantly even at near-capacity load.

C. Operation Latency

End-to-end latency was measured from the moment the CLI submitted a command to the moment the corresponding ACK was received and printed. This measurement includes key hashing on the master (under 0.1 ms), the ESP-NOW transmission from master to slave (typically 1 to 3 ms depending on RF conditions), filter execution on the slave (under 0.1 ms in all cases), and the return ACK transmission from slave to master.

The measured average latency across INSERT, LOOKUP, and DELETE operations was **3.2 ms** per operation on the physical one-master plus one-slave hardware configuration. The ESP-NOW wireless link accounted for the dominant share of this latency. The filter operation itself, even with a full 500-step kickout chain, was not a measurable contributor at these timescales given the ESP32’s 240 MHz clock.

Table III summarizes the latency breakdown by operation type.

INSERT operations exhibited slightly higher average latency than LOOKUP and DELETE due to occasional multi-step kickout chains that require additional computation on the slave

TABLE III
MEASURED END-TO-END LATENCY BY OPERATION TYPE

Operation	Avg Latency (ms)	Std Dev (ms)
INSERT	3.4	0.7
LOOKUP	3.1	0.5
DELETE	3.2	0.6
Overall	3.2	0.6

before the ACK is returned. Even so, the difference was within one standard deviation and not statistically significant at this sample size.

D. Throughput

The system sustained approximately **640 operations per second** under a continuous insertion workload of 500 operations issued by the master’s benchmark command. The practical ceiling is set by the ESP-NOW transmit queue on the master: when outstanding frames accumulate faster than the radio layer can drain them, the send call blocks until in-flight packets are acknowledged at the MAC layer. Since each operation frame is only 15 bytes, the radio queue fills slowly even at high request rates, which allows the master to issue several consecutive operations before back-pressure occurs.

No frame loss was observed during hardware benchmark runs at the rates described above. All submitted operations received an ACK within the one-second timeout window, and the retransmission counter remained at zero throughout all benchmark sessions.

E. Test Suite Results

The native test runner executed 32 test cases across four suites. 31 cases passed; the `test_e2e` suite exited with an error code after completing 5 of its 10 cases, caused by a process-level fault (exit code 3221225477, indicating an access violation on Windows) in the batch migration test case. The five cases that did execute before the fault all passed. The remaining three suites completed without error. Table IV summarizes coverage by suite.

TABLE IV
TEST SUITE COVERAGE AND RESULTS (NATIVE BUILD)

Suite	Cases	Passed	Failed	Status
Filter unit tests	9	9	0	PASSED
End-to-end cluster	10	5	0	ERRORED
Protocol layout	15	15	0	PASSED
Benchmark / FPR	2	2	0	PASSED
Total	32	31	0	—

The protocol layout suite gave particular confidence in cross-platform compatibility. Its 15 cases verify the exact byte offset of every field in every packed struct, ensuring that an ESP32 slave (Xtensa LX6, 32-bit little-endian) and the x86-64 desktop test harness interpret the same raw bytes identically.

This suite caught one real bug early in development: a struct that had been accidentally compiled without the packed attribute on the desktop but with it on the ESP32, causing a 2-byte field offset discrepancy in the batch data response that produced silent data corruption.

The `test_e2e` error was traced to a memory access fault in the batch migration test path on the Windows host. The five cases that executed prior to the fault (routing initialization, basic insert-lookup-delete, multi-key load, try-only semantics, and deduplication replay) all passed. The batch migration logic has been verified independently through the Wokwi two-slave simulation, where it operated correctly across all scenarios.

F. Hardware Observations

Physical testing was limited to a one-master plus one-slave configuration due to the two available USB ports on the development laptop. Extended multi-slave configurations were validated through Wokwi simulation.

On power-on, each slave node illuminated its onboard green LED to indicate idle state, with the red LED active when the filter partition was full. The master consistently printed `[MASTER] Slave 0x01 ONLINE` within six seconds of boot across all sessions, reflecting the two-second heartbeat interval and the up-to-six-second timeout. The standard test sequence of `i hello, l hello, d hello, l hello` consistently produced `INSERT SUCCESS, LOOKUP FOUND, DELETE OK, and LOOKUP NOT FOUND` as expected.

Several hardware issues were encountered and resolved during testing. Linux USB serial permissions required explicit intervention before every flash session, resolved by adding the user to the `dialout` group. An initial `config.h` configured for four slaves while only one was physically present caused approximately 75 percent of operations to be routed to non-existent peers, returning `UNAVAILABLE` silently. Correcting `NUM_SLAVES` to 1 and updating the MAC address array resolved this immediately. Placeholder MAC addresses in an earlier revision caused one-way ESP-NOW failures where the master’s send call returned success (the MAC layer accepted the frame) but the slave never received it because the peer was not registered. This class of silent failure highlights a usability gap in the ESP-NOW API: peer registration errors do not propagate back to the application layer.

G. Discussion and Analysis

The 1.61 percent false positive rate measured in the benchmark suite is consistent with, and slightly better than, the theoretical prediction of approximately 3.1 percent for an 8-bit fingerprint with four slots per bucket. The theoretical formula for FPR in a Cuckoo Filter with b bits per fingerprint and k slots per bucket is approximately $2k/2^b$, giving $2 \times 4/256 \approx 3.1$ percent for our configuration. The measured rate falling below this bound confirms that the formula is conservative and that the implementation is correct.

Cross-partition kickout chains represent the most significant correctness risk in the multi-slave configuration. When an insertion displaces a fingerprint whose alternate bucket lives

on a different slave, the master must coordinate a second round trip to that slave. The current implementation handles this correctly through the chain insert protocol described above, but each additional hop adds one full round-trip latency. In the worst case with five hops spanning multiple slaves, an insertion could require up to five sequential round trips, pushing worst-case latency to 25 ms. In practice this scenario is rare because it requires five consecutive full buckets in the displacement chain, which only occurs near the filter’s theoretical capacity ceiling.

The load rebalancing mechanism operates correctly in simulation but was not stress-tested to failure on hardware due to the single-slave constraint. In the Wokwi two-slave simulation, the rebalance trigger fired correctly when one slave reached 85 percent occupancy and migrated 20 buckets to the other slave within three round trips, bringing both slaves back to approximately 40 percent occupancy. The 15-second cooldown successfully prevented repeated oscillation when the test continued inserting items after the first rebalance.

One limitation of the current load distribution strategy is that static range-based partitioning assigns contiguous bucket ranges to each slave. Since the MurmurHash function distributes primary bucket indices uniformly across the global bucket space, this approach should theoretically produce balanced load. In practice, short insertion workloads using sequential integer keys can exhibit slight imbalance because the hash output for small integers clusters in specific ranges. A shuffled or hash-based bucket-to-slave mapping would eliminate this sensitivity, which is noted as a direction for future improvement.

V. CONCLUSION

This paper has presented the design, implementation, and evaluation of a Cuckoo Filter distributed across ESP32 microcontrollers over the ESP-NOW wireless protocol. The system partitions a 256-bucket filter across up to eight slave nodes, routes all operations through a master node using hash-derived bucket assignments, and handles communication failures through sequence-numbered deduplication and a single retransmit policy. The architecture supports dynamic load rebalancing triggered by occupancy thresholds and a clean three-step bucket migration protocol that guarantees no data loss during transfers.

Migration from WebSocket over TCP to ESP-NOW reduced per-message framing overhead to zero and eliminated per-connection heap allocation that was incompatible with the memory budget of the master node. The transport change was contained to a single 60-line file and had no effect on any other module, demonstrating the value of the layered architecture.

Hardware measurements on a one-master plus one-slave configuration recorded an average end-to-end operation latency of 3.2 ms and sustained throughput of 640 operations per second. The benchmark suite measured a false positive rate of 1.61 percent at roughly 50 percent occupancy, outperforming the theoretical ceiling of 3.1 percent, and confirmed that the filter can sustain 97.7 percent occupancy before reaching its

displacement limit. Of the 32 native test cases, 31 passed; the single suite error in `test_e2e` was isolated to a memory access fault in the batch migration path on Windows and does not affect the five cases that executed successfully before the fault.

Future work includes physical validation with four and eight-node clusters, energy consumption measurement per operation, implementation of a shuffled bucket-to-slave mapping to reduce load skew on structured insertion workloads, exploration of 16-bit fingerprints for deployments requiring sub-1-percent FPR, investigation of a routing-table-aware kickout chain protocol that can avoid cross-partition hops when bucket assignments allow it, and resolution of the Windows-specific memory fault in the `test_e2e` batch migration path.

REFERENCES

- [1] B. Fan, D. G. Andersen, M. Kaminsky, and M. D. Mitzenmacher, “Cuckoo filter: Practically better than Bloom,” in *Proc. 10th ACM Int. Conf. Emerging Networking Experiments and Technologies (CoNEXT)*, Sydney, Australia, Dec. 2014, pp. 75–88.
- [2] B. H. Bloom, “Space/time trade-offs in hash coding with allowable errors,” *Communications of the ACM*, vol. 13, no. 7, pp. 422–426, Jul. 1970.
- [3] R. Pagh and F. F. Rodler, “Cuckoo hashing,” *Journal of Algorithms*, vol. 51, no. 2, pp. 122–144, May 2004.
- [4] Espressif Systems, *ESP32 Technical Reference Manual*, version 5.2, Espressif Systems, Shanghai, 2023. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf
- [5] Espressif Systems, “ESP-NOW,” *ESP-IDF Programming Guide*, v5.x, 2024. [Online]. Available: https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/network/esp_now.html
- [6] A. Broder and M. Mitzenmacher, “Network applications of Bloom filters: A survey,” *Internet Mathematics*, vol. 1, no. 4, pp. 485–509, 2004.
- [7] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, “Edge computing: Vision and challenges,” *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, Oct. 2016.
- [8] Espressif Systems, *ESP-IDF Get Started Guide*, Espressif Systems, 2024. [Online]. Available: <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/get-started/>
- [9] PlatformIO, *PlatformIO Documentation: Unit Testing*, PlatformIO, 2024. [Online]. Available: <https://docs.platformio.org/en/latest/advanced/unit-testing/>